INTERNSHIP TASK REPORT

WEEK 01

DATA COLLECTION AND PREPROCESSING

Marketing Campaign Analytics



SUBMITTED BY

Khushi Singh

B.Tech – Computer Science and Engineering

NGF College of Engineering and Technology

TECHNOLOGIES USED

Python | Pandas | NumPy | Jupyter Notebook

SUBMISSION DATE

05 November 2026

TABLE OF CONTENTS

S. No.	Contents
1.	Introduction
2.	Project Objectives
3.	Dataset Description
4.	Tools and Technologies Used
5.	Data Inspection and Exploration
6.	Data Cleaning
7.	Data Transformation
8.	Feature Engineering
9.	Data Validation and Final Output
10.	Challenges and Learnings
11.	Conclusion

1. Introduction

In today's digital world, businesses use various online marketing channels to promote their products and services. These marketing activities generate large amounts of data, including impressions, clicks, conversions, campaign expenses, and customer engagement metrics.

Marketing analytics involves collecting, processing, and examining this data to understand campaign performance and support informed business decisions.

However, raw datasets often contain missing values, duplicate records, inconsistent entries, and formatting issues. Such problems can affect the accuracy and reliability of analysis. Therefore, data preprocessing is an essential step before performing any meaningful analysis.

This project focuses on the collection, cleaning, and preprocessing of marketing campaign data using Python and its Pandas library. The dataset includes information about different marketing campaigns, their advertising channels, performance metrics, and associated costs.

Throughout this task, various data cleaning and transformation techniques were applied to improve data quality and prepare the dataset for further analysis.

The final objective is to produce a structured and reliable dataset that can be used for future marketing performance analysis.

1.1 Importance of Data Preprocessing

Data preprocessing plays an important role in ensuring that a dataset is accurate, consistent, and suitable for analysis.

It helps in:

- Identifying and handling missing values.
- Detecting duplicate records.
- Correcting inconsistent data entries.
- Standardizing data formats and data types.
- Improving the overall quality and reliability of data.
- Preparing data for further statistical analysis and visualization.

2. Project Objectives and Scope

2.1 Project Objectives

The primary objective of this project is to perform data collection, cleaning, and preprocessing on a marketing campaign dataset using Python and the Pandas library.

The specific objectives are:

1. To understand the structure and characteristics of marketing campaign data.
2. To inspect the dataset and identify data quality issues.

3. To detect duplicate records and handle missing values.
4. To identify and correct inconsistent data entries.
5. To standardize column names, data types, and categorical values.
6. To convert date and numerical columns into appropriate formats.
7. To create additional analytical features, including campaign duration, Click-Through Rate (CTR), and conversion rate.
8. To validate the processed dataset and export it into CSV format.

2.2 Project Scope

The scope of this project covers the initial stages of a marketing analytics workflow, focusing primarily on improving the quality and structure of the available dataset.

The work includes:

- Dataset inspection and exploration.
- Data quality assessment.
- Data cleaning and standardization.
- Data type conversion.
- Feature engineering.
- Data validation.
- Exporting the final processed dataset.

The project is limited to data preparation and preprocessing. Detailed marketing performance analysis, predictive modelling, and visualization are considered potential future activities.

2.3 Expected Outcome

The expected outcome is a clean, structured, and analysis-ready marketing campaign dataset. The final output is intended to support further analysis of advertising performance, campaign engagement, spending, and conversions.

3. Dataset Description and Tools Used

3.1 Dataset Overview

For this project, a marketing campaign dataset was used to perform data cleaning and preprocessing operations.

The dataset contains information about different advertising campaigns conducted through various digital marketing channels. It includes campaign identifiers, campaign names, campaign dates, advertising platforms, impressions, clicks, campaign expenses, conversions, and campaign status.

The original dataset consisted of **100 records and 12 columns**. It was provided in CSV format and imported into Python using the Pandas library.

3.2 Dataset Attributes

The dataset contains the following important attributes:

Column Name	Description
Campaign_ID	Unique identifier for each campaign
Campaign_Name	Name assigned to the campaign
Start_Date	Campaign starting date
End_Date	Campaign ending date
Channel	Advertising platform used
Impressions	Number of times an advertisement was displayed
Clicks	Number of clicks received
Spend	Amount spent on the campaign
Conversions	Number of successful conversions
Active	Indicates whether the campaign is active
Campaign_Tag	Short identifier associated with the campaign

3.3 Dataset Structure

The dataset was initially loaded into a Pandas DataFrame for inspection and further processing.

A DataFrame is a two-dimensional data structure consisting of rows and columns, similar to a spreadsheet.

3.4 Tools and Technologies Used

The following tools and libraries were used:

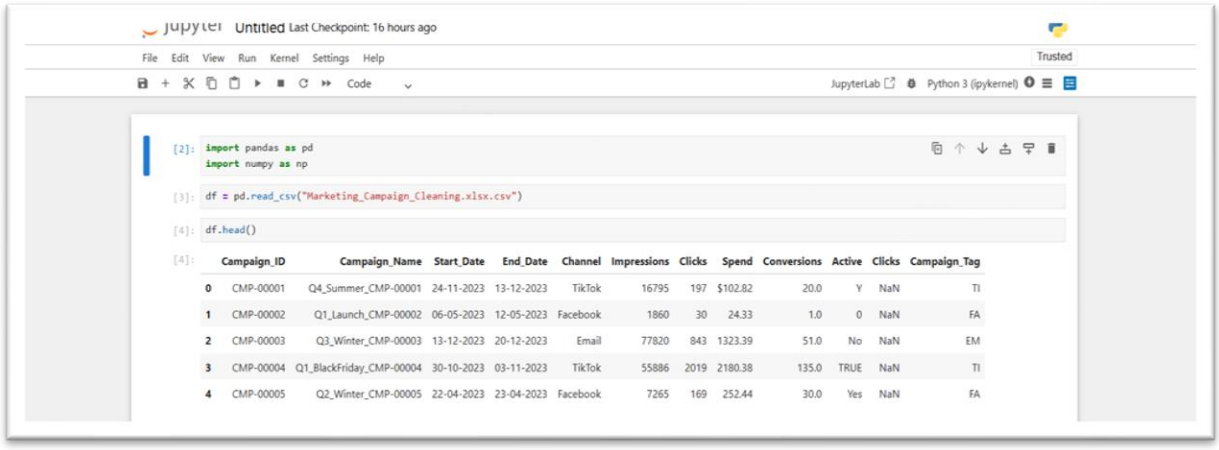
Python: Used as the primary programming language for data processing.

Pandas: Used for dataset loading, inspection, cleaning, transformation, and exporting.

NumPy: Used for numerical operations and handling conditional calculations.

Jupyter Notebook: Used as the development environment for writing and executing Python code.

Microsoft Excel: Used for viewing and inspecting CSV data.



```
[2]: import pandas as pd
     import numpy as np

[3]: df = pd.read_csv("Marketing_Campaign_Cleaning.xlsx.csv")

[4]: df.head()
```

[4]:

	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Clicks	Campaign_Tag
0	CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	$102.82	20.0	Y	NaN	TI
1	CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1.0	0	NaN	FA
2	CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51.0	No	NaN	EM
3	CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135.0	TRUE	NaN	TI
4	CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30.0	Yes	NaN	FA

Figure 1: Initial preview of the marketing campaign dataset using Pandas.



```
[5]: df.shape

[5]: (100, 12)

[6]: df.columns

[6]: Index([' Campaign_ID ', 'Campaign_Name', 'Start_Date', 'End_Date', 'Channel',
            'Impressions', 'Clicks ', 'Spend', 'Conversions', 'Active', 'Clicks',
            'Campaign_Tag'],
           dtype='str')

[7]: df.info()
```

Figure 2: Dimensions of the original marketing campaign dataset.

4.Data Inspection and Exploration

Before performing data cleaning and preprocessing, an initial inspection of the dataset was carried out using Pandas. The purpose of this step was to understand the dataset's structure, identify the available attributes, examine data types, and detect potential data quality issues.

Various Pandas functions were used to explore the dataset and evaluate its overall condition before applying cleaning techniques.

This all helped in analysing data and cleaning in data.

4.1 Displaying Initial Records

Code used: df.head()

The `head()` function was used to display the first five records of the dataset. This helped in understanding the arrangement of columns, the type of information stored, and the general structure of the campaign records.

4.2 Checking Dataset Dimensions

Code used: df.shape

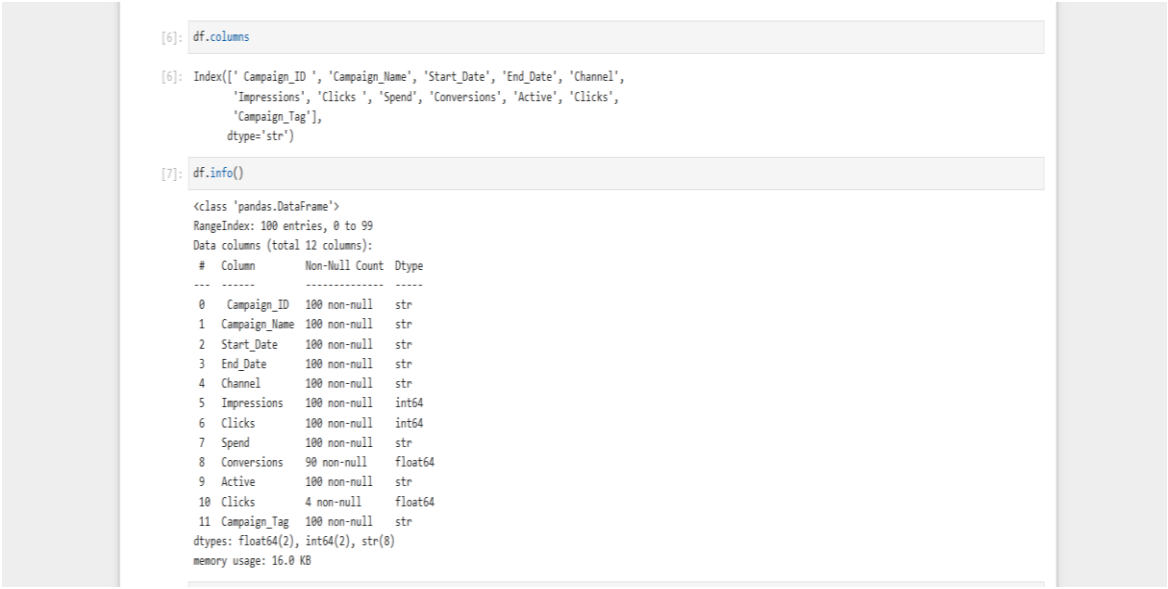
The `shape` attribute was used to identify the total number of rows and columns present in the dataset. The initial dataset contained 100 rows and 12 columns, providing an overview of its size and structure.

4.3 Checking Column Names and Data Types

Code used: df.columns
df.info()

The `columns` attribute was used to examine the names of all available columns. The `info()` function provided information about column data types, non-null values, and memory usage.

This inspection helped identify the different types of information stored in the dataset and determine which columns required data type conversion during preprocessing.



```
[6]: df.columns

[6]: Index([' Campaign_ID ', 'Campaign_Name', 'Start_Date', 'End_Date', 'Channel',
       'Impressions', 'Clicks ', 'Spend', 'Conversions', 'Active', 'Clicks',
       'Campaign_Tag'],
      dtype='str')

[7]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 12 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0    Campaign_ID   100 non-null    str
 1   Campaign_Name  100 non-null    str
 2   Start_Date     100 non-null    str
 3   End_Date       100 non-null    str
 4   Channel        100 non-null    str
 5   Impressions    100 non-null    int64
 6   Clicks         100 non-null    int64
 7   Spend          100 non-null    str
 8   Conversions    90 non-null     float64
 9   Active         100 non-null    str
 10  Clicks         4 non-null      float64
 11  Campaign_Tag   100 non-null    str
dtypes: float64(2), int64(2), str(8)
memory usage: 16.0 KB
```

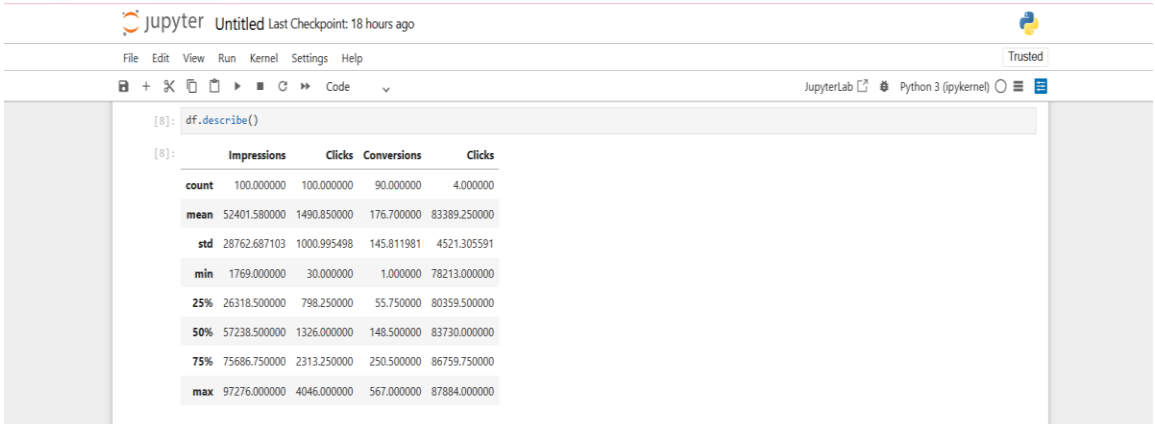
Figure 5: Dataset information showing column names, data types, and non-null counts

4.4 Statistical Summary

Code used: df.describe()

The `describe()` function was used to generate a statistical summary of the numerical columns. It provided information such as count, mean, standard deviation, minimum, maximum, and quartile values.

This helped in understanding the numerical distribution of campaign metrics such as impressions, clicks, spend, and conversions.



	Impressions	Clicks	Conversions	Clicks
count	100.000000	100.000000	90.000000	4.000000
mean	52401.580000	1490.850000	176.700000	83389.250000
std	28762.687103	1000.995498	145.811981	4521.305591
min	1769.000000	30.000000	1.000000	78213.000000
25%	26318.500000	798.250000	55.750000	80359.500000
50%	57238.500000	1326.000000	148.500000	83730.000000
75%	75686.750000	2313.250000	250.500000	86759.750000
max	97276.000000	4046.000000	567.000000	87884.000000

Figure 6: Statistical summary of numerical attributes.

4.5 Identifying Missing Values

Code used: df.isnull().sum()

The `isnull().sum()` function was used to identify the number of missing values in each column. This step helped locate incomplete records and determine which attributes required attention during the data cleaning process.

The inspection revealed missing information in certain columns, particularly the campaign conversion data and the additional duplicate-named Clicks column.

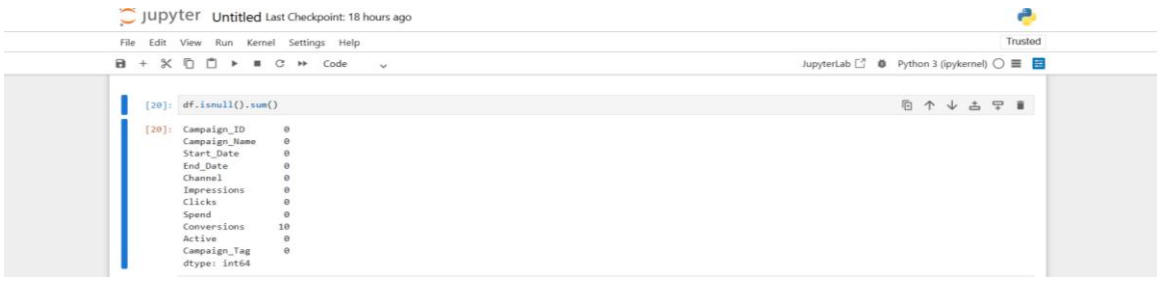


Figure 7: Identification of missing values in the original dataset.

4.6 Checking Duplicate Records

Code used: df.duplicated().sum()

The `duplicated().sum()` function was used to identify completely duplicated rows in the dataset. This check was performed to ensure that repeated records did not affect the reliability of subsequent analysis.

No exact duplicate rows were identified in the original dataset.

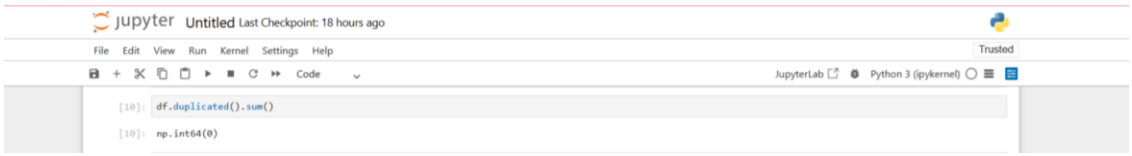


Figure 8: Duplicate record inspection.

5. DATA CLEANING

Data cleaning is an essential step in the data preprocessing process. It involves identifying and correcting errors, inconsistencies, and incomplete information within a dataset.

After inspecting the original marketing campaign dataset, several data quality issues were identified. These included unnecessary spaces in column names, duplicate-named columns, missing values, inconsistent channel names, and variations in campaign status.

Various Pandas functions were applied to address these issues and improve the consistency and reliability of the dataset.

5.1 Removing Unnecessary Spaces from Column Names

Code used: df.columns = df.columns.str.strip()

The `str.strip()` method was applied to remove unnecessary leading and trailing spaces from column names.This ensured consistent column naming and made it easier to access and manipulate the dataset attributes using Pandas

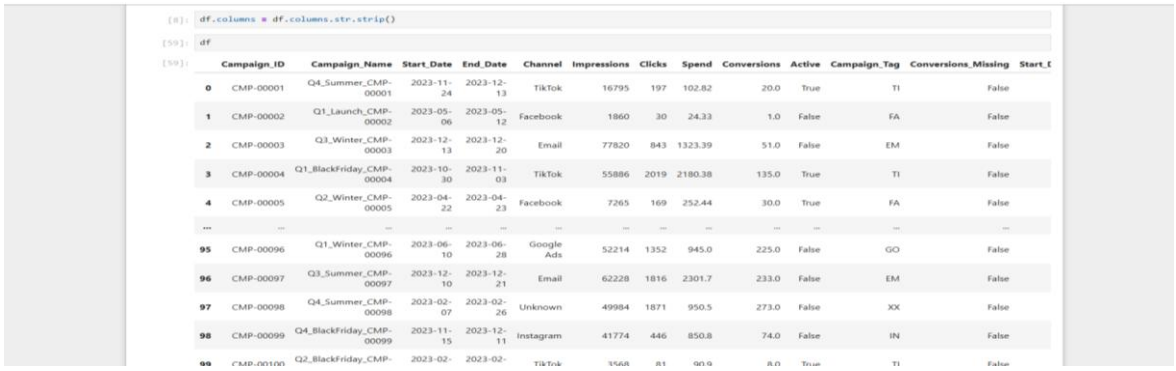


Figure 9: Removal of unnecessary spaces from column names.

5.2 Handling Duplicate-Named Columns

During the inspection process, two columns with the name `Clicks` were identified after removing surrounding spaces.

The primary Clicks column contained campaign click information for all 100 records, whereas the additional column contained only four non-missing values.

To maintain consistency and avoid ambiguity during analysis, the additional duplicate-named column was excluded, while the primary Clicks column was retained.

The remaining relevant columns were selected to create a structured DataFrame.

Code used: df = df.iloc[:, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11]].copy()

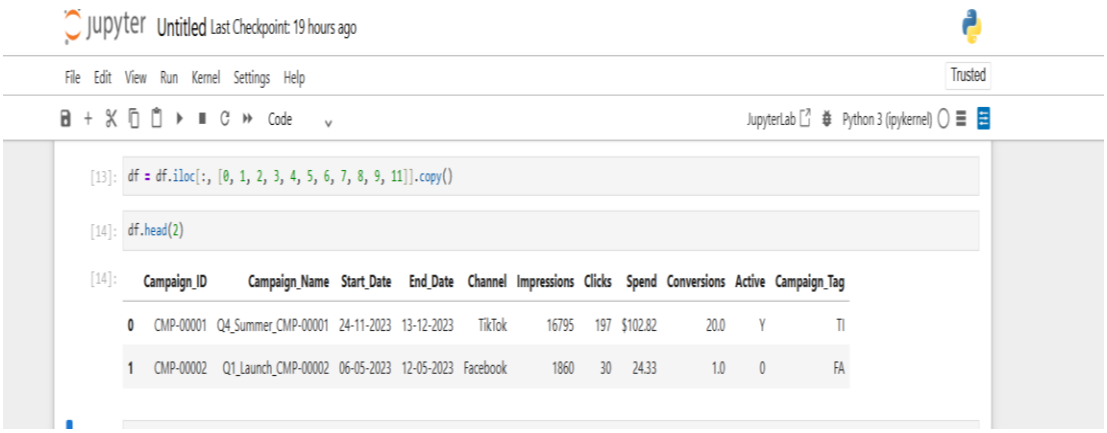


Figure 10: Selection of relevant columns and removal of the additional Clicks column.

5.3 Checking and Removing Duplicate Records

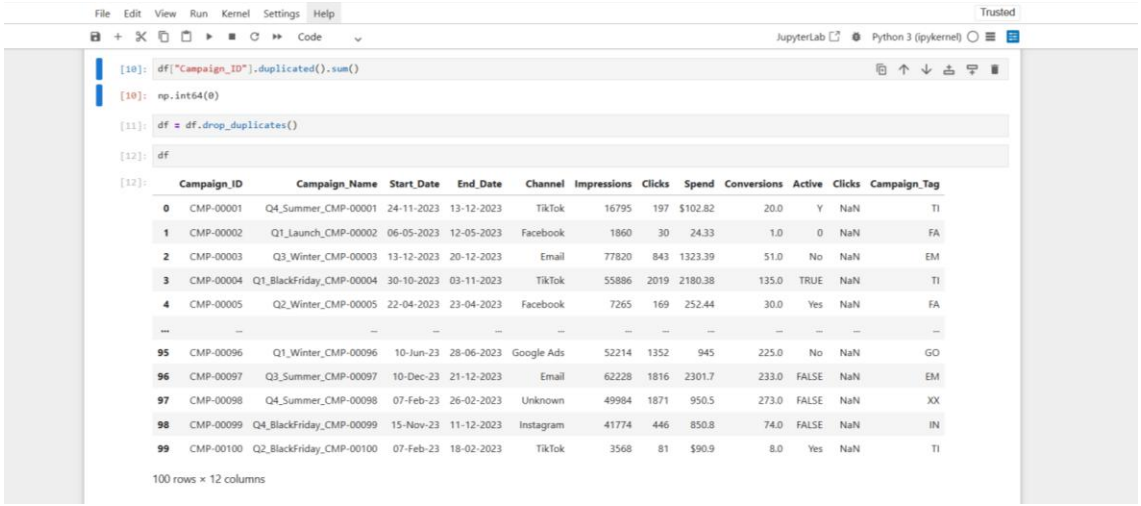
Code used: df.duplicated().sum()

df = df.drop_duplicates()

Duplicate records were checked using the `duplicated()` function. The inspection showed that no completely duplicated rows were present in the original dataset.

The `drop_duplicates()` method was also applied to ensure that any exact duplicate records would be removed if present.

As no exact duplicates were identified, the total number of campaign records remained unchanged.



```
[10]: df["Campaign_ID"].duplicated().sum()
[10]: np.int64(0)
[11]: df = df.drop_duplicates()
[12]: df
```

[12]:

	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Clicks	Campaign_Tag
0	CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	$102.82	20.0	Y	NaN	TI
1	CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1.0	0	NaN	FA
2	CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51.0	No	NaN	EM
3	CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135.0	TRUE	NaN	TI
4	CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30.0	Yes	NaN	FA
...	...	...	...	...	...	...	...	...	...	...	...	...
95	CMP-00096	Q1_Winter_CMP-00096	10-Jun-23	28-06-2023	Google Ads	52214	1352	945	225.0	No	NaN	GO
96	CMP-00097	Q3_Summer_CMP-00097	10-Dec-23	21-12-2023	Email	62228	1816	2301.7	233.0	FALSE	NaN	EM
97	CMP-00098	Q4_Summer_CMP-00098	07-Feb-23	26-02-2023	Unknown	49984	1871	950.5	273.0	FALSE	NaN	XX
98	CMP-00099	Q4_BlackFriday_CMP-00099	15-Nov-23	11-12-2023	Instagram	41774	446	850.8	74.0	FALSE	NaN	IN
99	CMP-00100	Q2_BlackFriday_CMP-00100	07-Feb-23	18-02-2023	TikTok	3568	81	$90.9	8.0	Yes	NaN	TI

100 rows × 12 columns

Figure 11: Duplicate record identification and handling.

5.4 Handling Missing Values

Code used: df.isnull().sum()

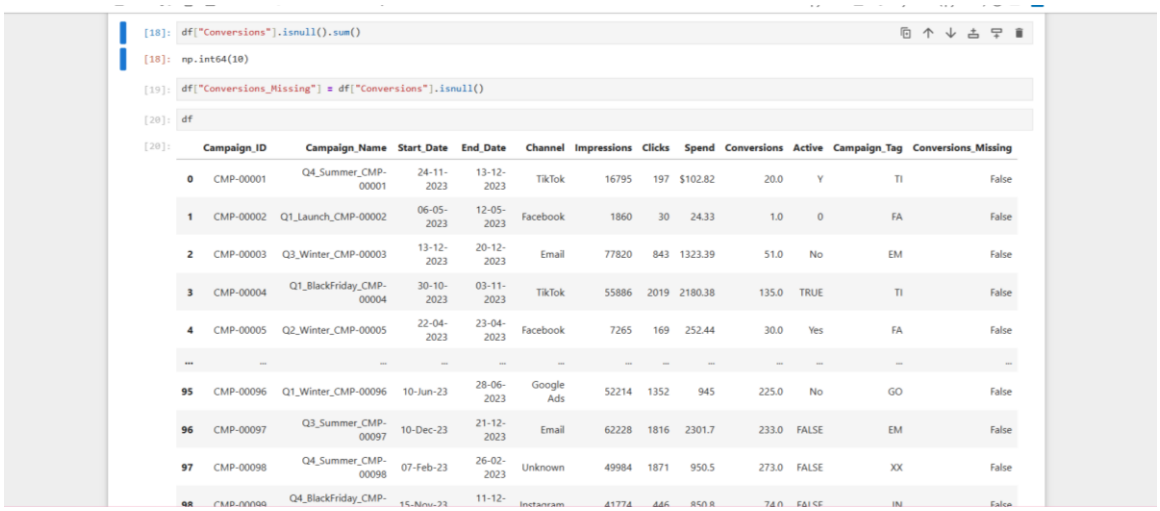
df["Conversions_Missing"] = df["Conversions"].isnull()

Missing values were identified using the `isnull().sum()` function.

The original dataset contained 10 missing values in the Conversions column. Instead of replacing these values with zero, they were preserved because missing conversion information does not necessarily indicate that no conversions occurred.

An additional column named `Conversions_Missing` was created to identify records where conversion information was unavailable.

This approach helped maintain the original information without introducing unsupported assumptions.



```
[18]: df["Conversions"].isnull().sum()
[18]: np.int64(10)
[19]: df["Conversions_Missing"] = df["Conversions"].isnull()
[20]: df
```

[20]:

	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing
0	CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	$102.82	20.0	Y	TI	False
1	CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1.0	0	FA	False
2	CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51.0	No	EM	False
3	CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135.0	TRUE	TI	False
4	CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30.0	Yes	FA	False
...	...	...	...	...	...	...	...	...	...	...	...	...
95	CMP-00096	Q1_Winter_CMP-00096	10-Jun-23	28-06-2023	Google Ads	52214	1352	945	225.0	No	GO	False
96	CMP-00097	Q3_Summer_CMP-00097	10-Dec-23	21-12-2023	Email	62228	1816	2301.7	233.0	FALSE	EM	False
97	CMP-00098	Q4_Summer_CMP-00098	07-Feb-23	26-02-2023	Unknown	49984	1871	950.5	273.0	FALSE	XX	False
98	CMP-00099	Q4_BlackFriday_CMP-	15-Nov-23	11-12-	Instagram	41774	446	850.8	74.0	FALSE	IN	False

Figure 12: Identification and handling of missing conversion values.

5.5 Correcting Inconsistent Channel Names

Code used: df["Channel"] = df["Channel"].replace({ "Gogle": "Google Ads"})

The Channel column was examined to identify inconsistent or incorrectly entered marketing platform names.

During inspection, the value `Gogle` was identified as a spelling inconsistency and corrected to `Google Ads`.

Records containing `Unknown` were retained because their actual marketing channels could not be determined from the available information.



Figure 13: Standardization of inconsistent marketing channel names.

5.6 Standardizing Campaign Status

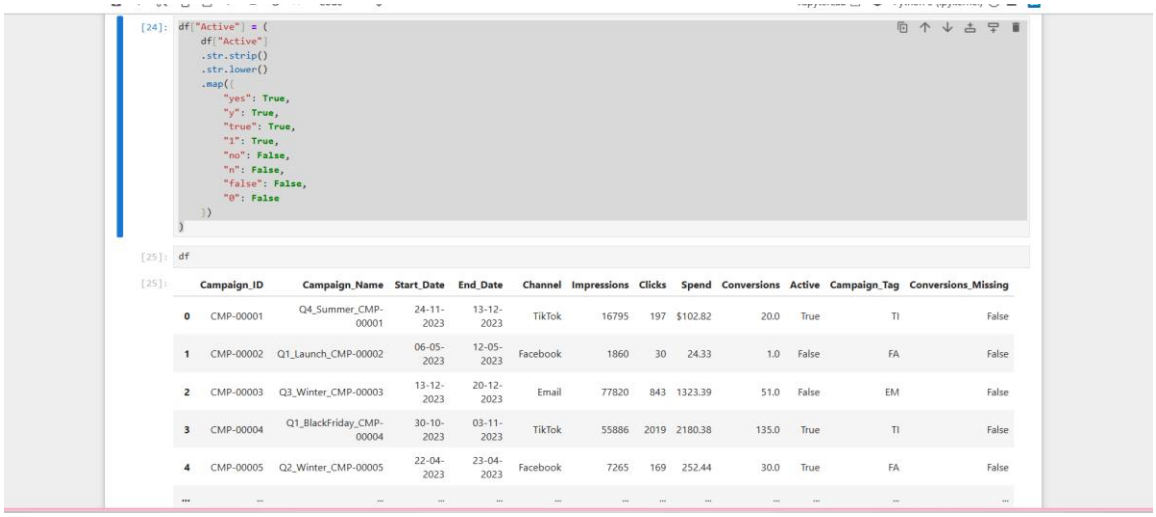
Code used: df["Active"] = df["Active"] .str.strip().str.lower().map({
"yes": True,
"y": True,
"true": True,
"1": True,
"no": False,
"n": False,
"false": False,
"0": False
}))

The Active column contained different representations of campaign status, including Yes, Y, TRUE, 1, No, and FALSE.

To maintain consistency, these values were standardized into Boolean format.

The values representing an active campaign were converted to `True`, while those representing an inactive campaign were converted to `False`.

This made the column more consistent and suitable for further analysis.



```
[24]: df["Active"] = (
          df["Active"]
          .str.strip()
          .str.lower()
          .map({
              "yes": True,
              "y": True,
              "true": True,
              "1": True,
              "no": False,
              "n": False,
              "false": False,
              "0": False
          })
      )

[25]: df
```

[25]:

	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing
0	CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	$102.82	20.0	True	TI	False
1	CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1.0	False	FA	False
2	CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51.0	False	EM	False
3	CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135.0	True	TI	False
4	CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30.0	True	FA	False
...	...	...	...	...	...	...	...	...	...	...	...	...

Figure 14: Standardization of campaign status into Boolean values.

6. DATA TRANSFORMATION

Data transformation is the process of converting data into a suitable format for analysis. After completing the initial data cleaning operations, different transformation techniques were applied to the marketing campaign dataset.

These operations included converting numerical values, standardizing date formats, handling invalid entries, and maintaining consistency across categorical attributes.

The transformations helped improve the structure and usability of the dataset.

Transformation Operations

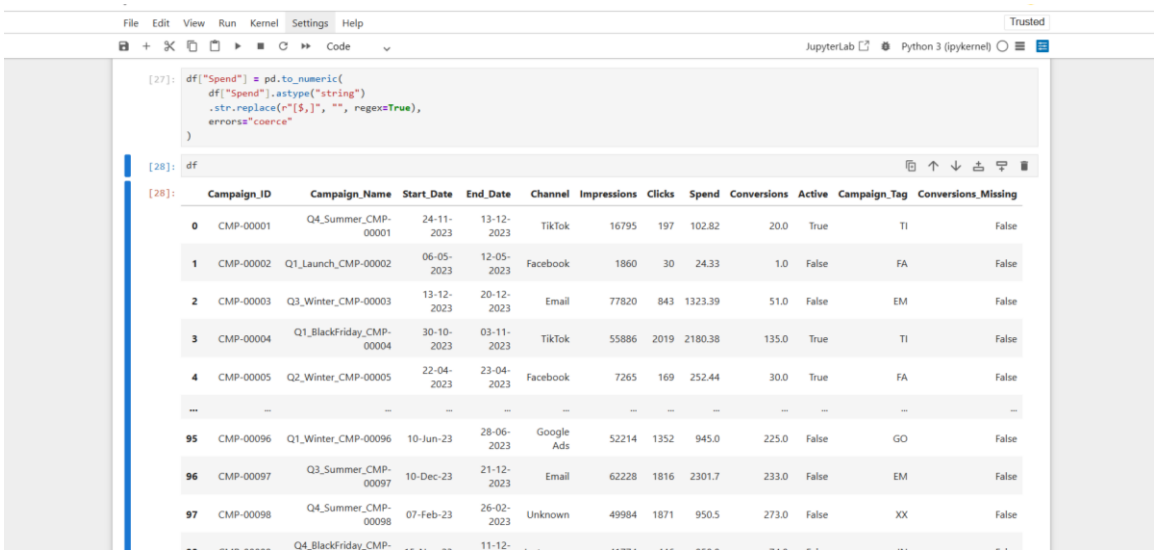
6.1 Converting Spend into Numerical Format

Code used:
```
df["Spend"] = pd.to_numeric(
    df["Spend"].astype("string")
    .str.replace(r"[$,]", "", regex=True),
     errors="coerce"
      )
```

The Spend column initially contained currency symbols along with numerical values. These symbols were removed using the `str.replace()` method.

The `pd.to_numeric()` function was then used to convert the cleaned values into a numerical data type.

This transformation made it possible to perform mathematical calculations, such as calculating total and average campaign expenditure.



	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing
0	CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	102.82	20.0	True	TI	False
1	CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1.0	False	FA	False
2	CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51.0	False	EM	False
3	CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135.0	True	TI	False
4	CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30.0	True	FA	False
...	...	...	...	...	...	...	...	...	...	...	...	...
95	CMP-00096	Q1_Winter_CMP-00096	10-Jun-23	28-06-2023	Google Ads	52214	1352	945.0	225.0	False	GO	False
96	CMP-00097	Q3_Summer_CMP-00097	10-Dec-23	21-12-2023	Email	62228	1816	2301.7	233.0	False	EM	False
97	CMP-00098	Q4_Summer_CMP-00098	07-Feb-23	26-02-2023	Unknown	49984	1871	950.5	273.0	False	XX	False

Figure 15: Conversion of campaign spending values into numerical format.

6.2 Converting Date Columns into Datetime Format

Code used:
```
df["Start_Date"] = pd.to_datetime(
    df["Start_Date"],
    format="mixed",
    dayfirst=True,
    errors="coerce"
    )

    df["End_Date"] = pd.to_datetime(
   df["End_Date"],
   format="mixed",
  dayfirst=True,
  errors="coerce"
  )
```

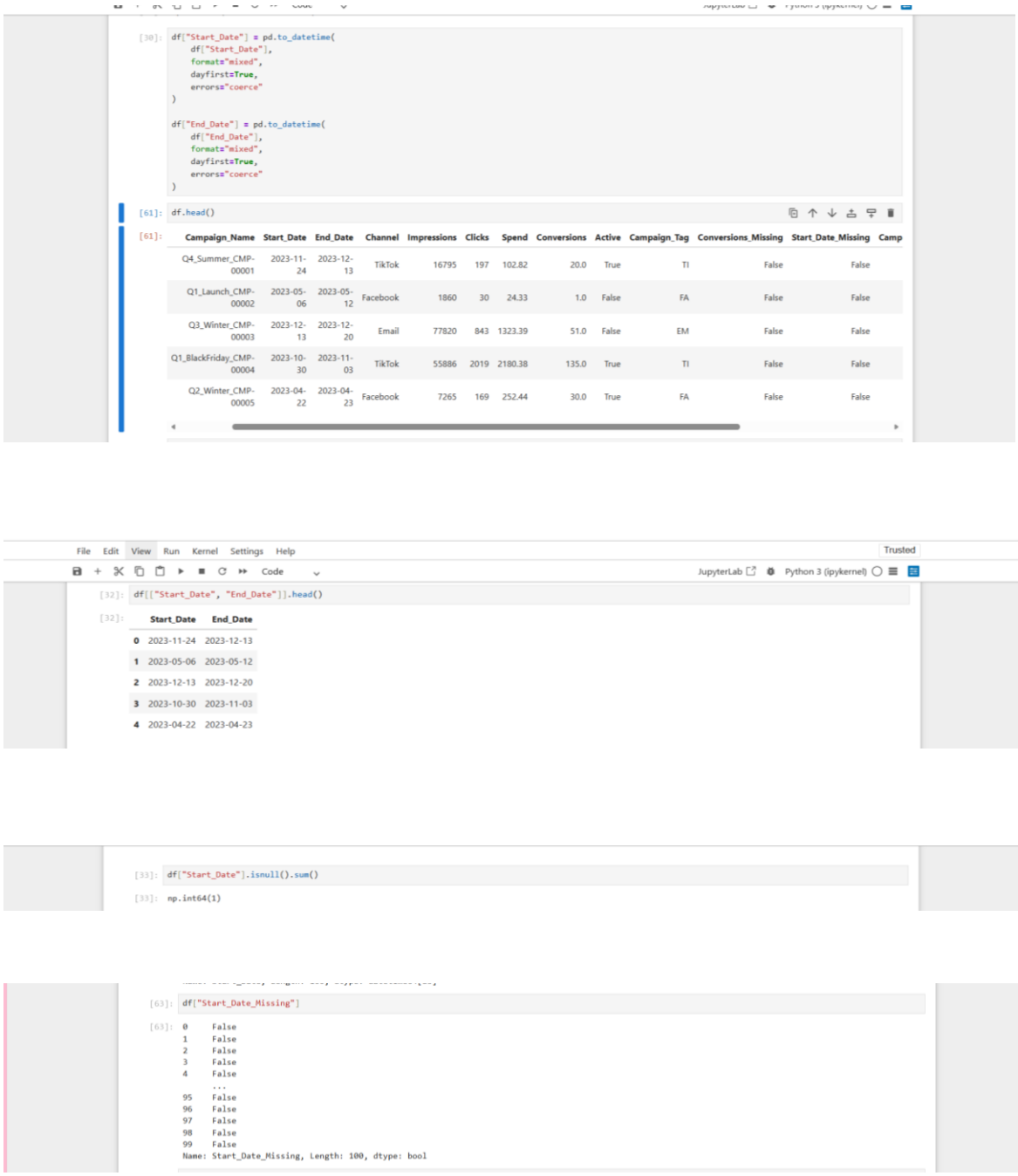
The Start_Date and End_Date columns contained date values in different formats.

The `pd.to_datetime()` function was used to convert these columns into a standardized datetime format.

The `format="mixed"` parameter allowed Pandas to interpret different date formats, while `dayfirst=True` specified day-first interpretation.

The `errors="coerce"` parameter converted invalid date entries into missing datetime values (NaT).

This transformation made the date columns suitable for date-based calculations and analysis.



```
[30]: df["Start_Date"] = pd.to_datetime(
          df["Start_Date"],
          format="mixed",
          dayfirst=True,
          errors="coerce"
      )

      df["End_Date"] = pd.to_datetime(
          df["End_Date"],
          format="mixed",
          dayfirst=True,
          errors="coerce"
      )

[61]: df.head()
```

[61]:

Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Date_Missing	Camp
Q4_Summer_CMP-00001	2023-11-24	2023-12-13	TikTok	16795	197	102.82	20.0	True	TI	False	False	
Q1_Launch_CMP-00002	2023-05-06	2023-05-12	Facebook	1860	30	24.33	1.0	False	FA	False	False	
Q3_Winter_CMP-00003	2023-12-13	2023-12-20	Email	77820	843	1323.39	51.0	False	EM	False	False	
Q1_BlackFriday_CMP-00004	2023-10-30	2023-11-03	TikTok	55886	2019	2180.38	135.0	True	TI	False	False	
Q2_Winter_CMP-00005	2023-04-22	2023-04-23	Facebook	7265	169	252.44	30.0	True	FA	False	False	

File Edit View Run Kernel Settings Help — Trusted

Code — JupyterLab — Python 3 (ipykernel)

```
[32]: df[["Start_Date", "End_Date"]].head()
```

[32]:

	Start_Date	End_Date
0	2023-11-24	2023-12-13
1	2023-05-06	2023-05-12
2	2023-12-13	2023-12-20
3	2023-10-30	2023-11-03
4	2023-04-22	2023-04-23


```
[33]: df["Start_Date"].isnull().sum()

[33]: np.int64(1)
```



```
[63]: df["Start_Date_Missing"]

[63]: 0     False
      1     False
      2     False
      3     False
      4     False
            ...
      95    False
      96    False
      97    False
      98    False
      99    False
      Name: Start_Date_Missing, Length: 100, dtype: bool
```

Figure 16: Conversion of campaign dates into datetime format.

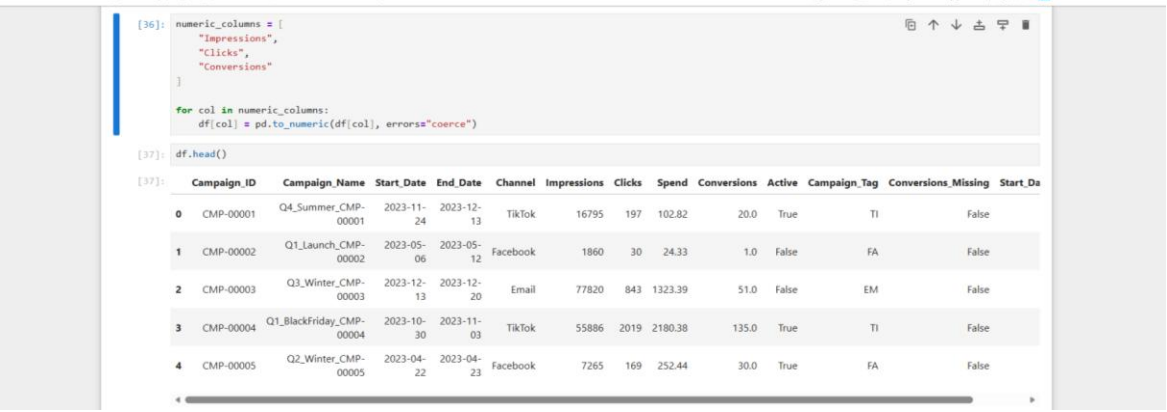
6.3 Converting Numerical Columns

Code used: numeric_columns = ["Impressions","Clicks","Conversions"]
for col in numeric_columns:
df[col] = pd.to_numeric(df[col], errors="coerce")

The Impressions, Clicks, and Conversions columns were converted into appropriate numerical data types using `pd.to_numeric()`.

A for loop was used to apply the conversion to multiple columns without repeating the same code.The `errors="coerce"` parameter ensured that any values that could not be converted into numbers would be represented as missing values.

This step prepared the numerical attributes for mathematical operations and further analysis.



```
[36]: numeric_columns = [
          "Impressions",
          "Clicks",
          "Conversions"
      ]

      for col in numeric_columns:
          df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
[37]: df.head()
```

[37]:

	Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Da
0	CMP-00001	Q4_Summer_CMP-00001	2023-11-24	2023-12-13	TikTok	16795	197	102.82	20.0	True	TI	False	
1	CMP-00002	Q1_Launch_CMP-00002	2023-05-06	2023-05-12	Facebook	1860	30	24.33	1.0	False	FA	False	
2	CMP-00003	Q3_Winter_CMP-00003	2023-12-13	2023-12-20	Email	77820	843	1323.39	51.0	False	EM	False	
3	CMP-00004	Q1_BlackFriday_CMP-00004	2023-10-30	2023-11-03	TikTok	55886	2019	2180.38	135.0	True	TI	False	
4	CMP-00005	Q2_Winter_CMP-00005	2023-04-22	2023-04-23	Facebook	7265	169	252.44	30.0	True	FA	False	

Figure 17: Conversion of campaign metrics into numerical data types.

6.4 Standardizing Campaign Tags

Code used:
```
tag_map = {
    "TikTok": "TI",
    "Facebook": "FA",
    "Email": "EM",
    "Instagram": "IN",
    "Google Ads": "GO",
    "Unknown": "XX"
}

df["Campaign_Tag"] = (
    df["Channel"].map(tag_map)
    .fillna(df["Campaign_Tag"])
)
```

A mapping dictionary was created to maintain consistent campaign tags according to the corresponding marketing channels.

The `map()` function was used to replace channel names with their respective short codes. The `fillna()` method retained the existing campaign tag wherever a channel did not have a matching mapping.This helped maintain consistency between the Channel and Campaign_Tag columns.

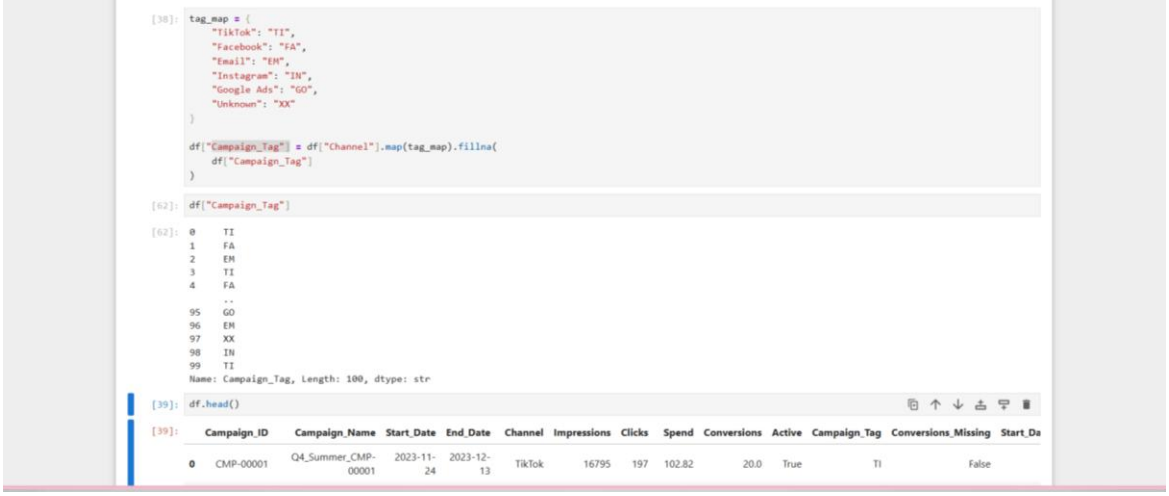


Figure 18: Standardization of campaign tags using a mapping dictionary.

6.5 Identifying Missing Date Information

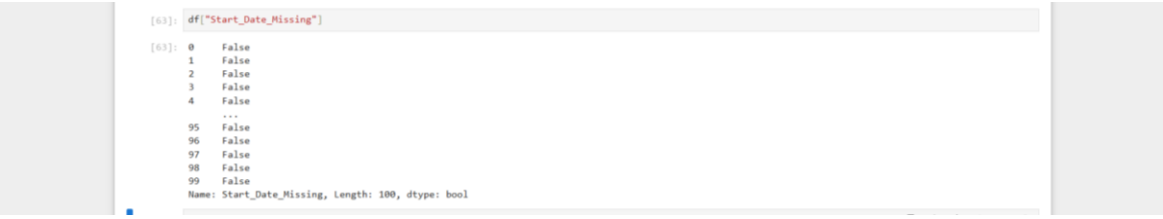
Code used: df["Start_Date_Missing"] = (df["Start_Date"].isnull())

After converting the date columns, an additional indicator column named `Start_Date_Missing` was created.

This column identifies records where the campaign start date was unavailable or invalid.The original missing information was preserved rather than being replaced with an estimated date. This helped maintain transparency and accuracy during preprocessing.



Figure 19: Identification of missing campaign start dates.



"Start_Date_Missing"

7.Feature Engineering

1. Introduction to Feature Engineering

Feature engineering is the process of creating new columns or features from existing data. It helps make the dataset more informative and useful for further analysis.

In this project, three important features were created:

1. Campaign Duration
2. Click-Through Rate (CTR)
3. Conversion Rate

2. Campaign Duration Calculation

Campaign duration represents the total number of days a marketing campaign was active.

Code used:
```
df["Campaign_Duration_Days"] = (
    df["End_Date"] - df["Start_Date"]).dt.days
    df["Campaign_Duration_Days"]

    df.loc[ df["Campaign_Duration_Days"] < 0,
     "Campaign_Duration_Days" ] = np.nan
```

- The end date is subtracted from the start date.
- `.dt.days` extracts the duration in days.
- Negative durations are treated as invalid and replaced with missing values.

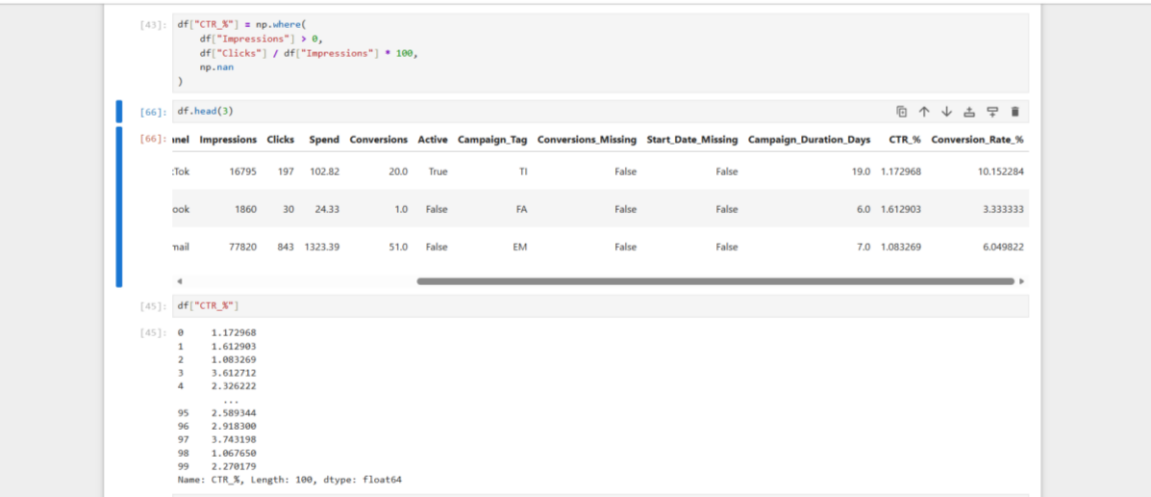


3. Click-Through Rate (CTR)

Click-Through Rate measures the percentage of people who clicked on an advertisement compared with the number of times it was displayed.

Code used:
```
df["CTR_%"] = np.where(
df["Impressions"] > 0,
df["Clicks"] / df["Impressions"] * 100,
np.nan
)
```

- Clicks are divided by impressions.
- The result is multiplied by 100 to calculate the percentage.
- If impressions are zero or less, the result is kept as missing to avoid invalid calculations.



```
[43]: df["CTR_%"] = np.where(
          df["Impressions"] > 0,
          df["Clicks"] / df["Impressions"] * 100,
          np.nan
      )

[66]: df.head(3)
```

[66]:	nel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Date_Missing	Campaign_Duration_Days	CTR_%	Conversion_Rate_%
	:Tok	16795	197	102.82	20.0	True	TI	False	False	19.0	1.172968	10.152284
	ook	1860	30	24.33	1.0	False	FA	False	False	6.0	1.612903	3.333333
	nail	77820	843	1323.39	51.0	False	EM	False	False	7.0	1.083269	6.049822

```
[45]: df["CTR_%"]

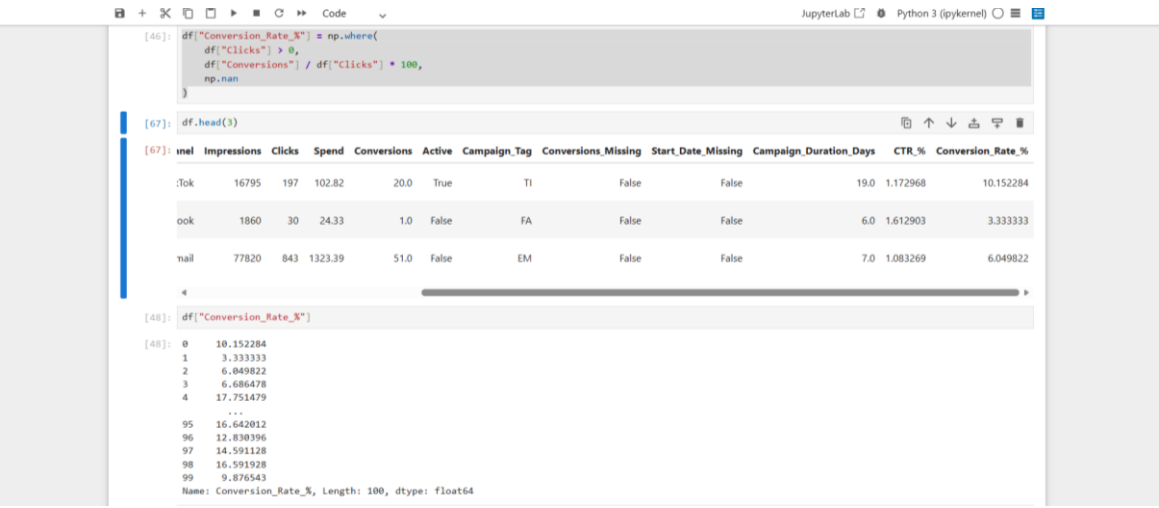
[45]: 0     1.172968
      1     1.612903
      2     1.083269
      3     3.612712
      4     2.326222
              ...
      95    2.589344
      96    2.918300
      97    3.743198
      98    1.067650
      99    2.270179
      Name: CTR_%, Length: 100, dtype: float64
```

4. Conversion Rate Calculation

Conversion Rate measures the percentage of clicks that resulted in conversions.

Code used:
```
df["Conversion_Rate_%"] = np.where(
df["Clicks"] > 0,
df["Conversions"] / df["Clicks"] * 100,
np.nan
)
```

- Conversions are divided by clicks.
- The result is multiplied by 100.
- If clicks are zero or less, the result is kept as missing.
- Missing conversion values remain missing rather than being treated as zero.



```
[46]: df["Conversion_Rate_%"] = np.where(
          df["Clicks"] > 0,
          df["Conversions"] / df["Clicks"] * 100,
          np.nan
      )

[67]: df.head(3)
```

[67]:

nel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Date_Missing	Campaign_Duration_Days	CTR_%	Conversion_Rate_%
:Tok	16795	197	102.82	20.0	True	TI	False	False	19.0	1.172968	10.152284
ook	1860	30	24.33	1.0	False	FA	False	False	6.0	1.612903	3.333333
nail	77820	843	1323.39	51.0	False	EM	False	False	7.0	1.083269	6.049822

```
[48]: df["Conversion_Rate_%"]
```

[48]:

```
0     10.152284
1      3.333333
2      6.049822
3      6.686478
4     17.751479
        ...
95    16.642012
96    12.830396
97    14.591128
98    16.591928
99     9.876543
Name: Conversion_Rate_%, Length: 100, dtype: float64
```

8.Data Validation and Final Output

1. Data Validation

Data validation is an important step in data preprocessing. It helps ensure that the cleaned dataset contains consistent values, appropriate data types, and no unnecessary duplicate records.

After completing the cleaning and transformation steps, the following checks were performed:

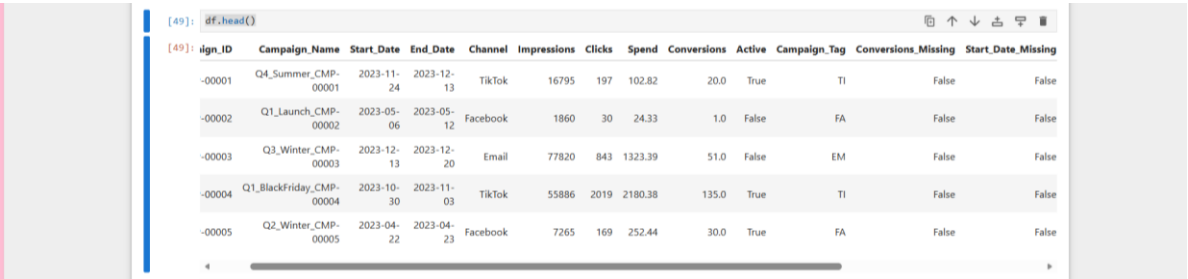
1. Column Validation: Verified that the dataset contained the required columns.
2. Duplicate Validation: Checked for duplicate records.
3. Data Type Validation: Ensured that numerical and date columns had appropriate data types.
4. Missing Value Validation: Checked missing values in the cleaned dataset.
5. Logical Validation: Checked campaign duration to identify invalid negative values.

2. Checking the Final Dataset

The following Pandas commands were used to review the cleaned dataset.

Python code:

df.head()



```
[49]: df.head()
```

[49]:

ign_ID	Campaign_Name	Start_Date	End_Date	Channel	Impressions	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Date_Missing
-00001	Q4_Summer_CMP-00001	2023-11-24	2023-12-13	TikTok	16795	197	102.82	20.0	True	TI	False	False
-00002	Q1_Launch_CMP-00002	2023-05-06	2023-05-12	Facebook	1860	30	24.33	1.0	False	FA	False	False
-00003	Q3_Winter_CMP-00003	2023-12-13	2023-12-20	Email	77820	843	1323.39	51.0	False	EM	False	False
-00004	Q1_BlackFriday_CMP-00004	2023-10-30	2023-11-03	TikTok	55886	2019	2180.38	135.0	True	TI	False	False
-00005	Q2_Winter_CMP-00005	2023-04-22	2023-04-23	Facebook	7265	169	252.44	30.0	True	FA	False	False

df.shape

```
[50]: df.shape
[50]: (100, 16)
```

df.info()

```
[51]: df.info()
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 16 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Campaign_ID            100 non-null    str
 1   Campaign_Name          100 non-null    str
 2   Start_Date             99 non-null     datetime64[us]
 3   End_Date               100 non-null    datetime64[us]
 4   Channel                100 non-null    str
 5   Impressions            100 non-null    int64
 6   Clicks                 100 non-null    int64
 7   Spend                  100 non-null    Float64
 8   Conversions            90 non-null     float64
 9   Active                 100 non-null    bool
 10  Campaign_Tag           100 non-null    str
 11  Conversions_Missing    100 non-null    bool
 12  Start_Date_Missing     100 non-null    bool
 13  Campaign_Duration_Days  96 non-null    float64
 14  CTR_%                  100 non-null    float64
 15  Conversion_Rate_%      90 non-null     float64
dtypes: Float64(1), bool(3), datetime64[us](2), float64(4), int64(2), str(4)
memory usage: 14.5 KB
```

df.isnull().sum()

```
[52]: df.isnull().sum()
[52]: Campaign_ID              0
      Campaign_Name            0
      Start_Date               1
      End_Date                 0
      Channel                  0
      Impressions              0
      Clicks                   0
      Spend                    0
      Conversions             10
      Active                   0
      Campaign_Tag             0
      Conversions_Missing      0
      Start_Date_Missing       0
      Campaign_Duration_Days   4
      CTR_%                    0
      Conversion_Rate_%       10
      dtype: int64
```

df["Channel"].value_counts()

```
[53]: df["Channel"].value_counts()
[53]: Channel
      Email         26
      Google Ads    21
      Instagram     20
      TikTok        16
      Facebook      13
      Unknown        4
      Name: count, dtype: int64
```

df["Active"].value_counts(dropna=False)

```
[54]: df["Active"].value_counts(dropna=False)
[54]: Active
      True     62
      False    38
      Name: count, dtype: int64
```

df.duplicated().sum()

```
[56]: df.duplicated().sum()
[56]: np.int64(0)
```

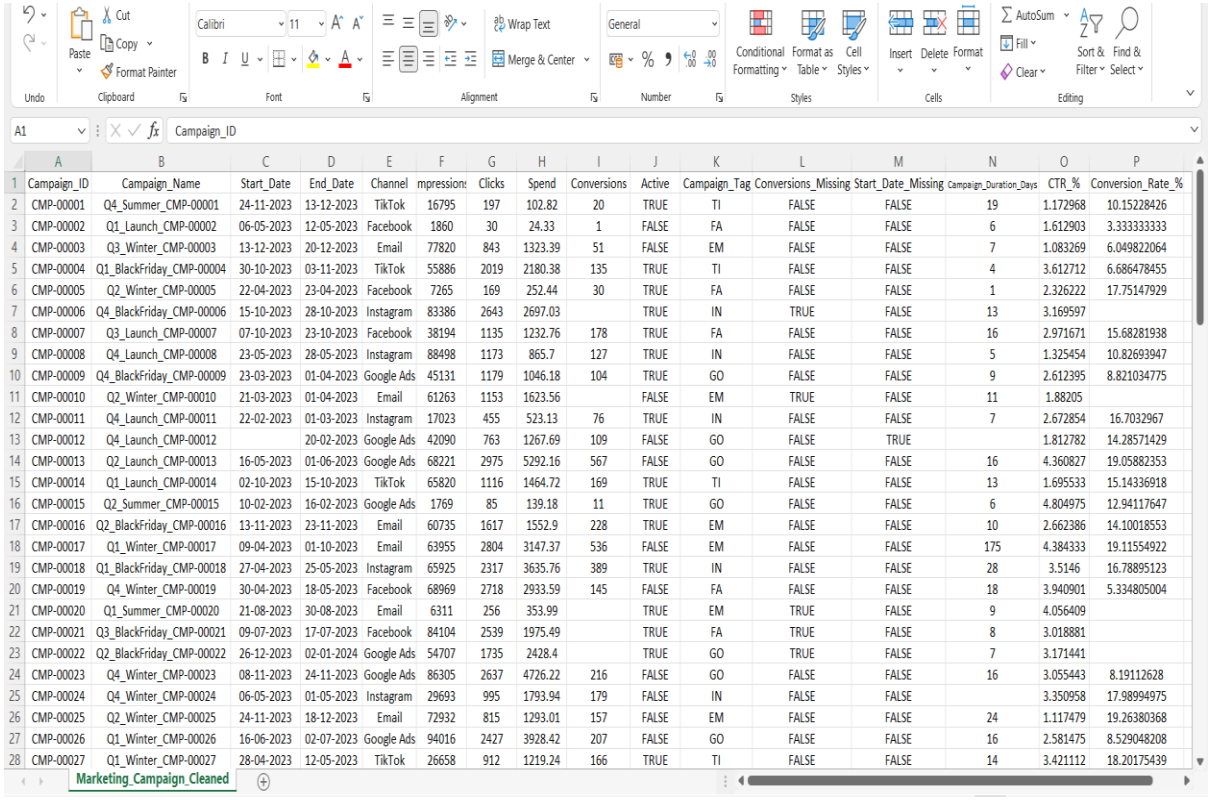
df.describe()

```
[55]: df.describe()
```

[55]:

	Start_Date	End_Date	Impressions	Clicks	Spend	Conversions	Campaign_Duration_Days	CTR_%	Conversion_Rate_%
count	99	100	100.000000	100.000000	100.0	90.000000	96.000000	100.000000	90.000000
mean	2023-07-10 19:38:10.909091	2023-07-27 05:16:48	52401.580000	1490.850000	1942.8039	176.700000	18.708333	2.829166	11.609623
min	2023-01-01 00:00:00	2023-01-17 00:00:00	1769.000000	30.000000	24.33	1.000000	1.000000	1.034670	3.333333
25%	2023-04-21 00:00:00	2023-05-09 06:00:00	26318.500000	798.250000	657.915	55.750000	8.000000	1.993689	7.102281
50%	2023-07-09 00:00:00	2023-08-07 12:00:00	57238.500000	1326.000000	1741.635	148.500000	14.000000	2.786240	10.738914
75%	2023-10-09 12:00:00	2023-10-28 06:00:00	75686.750000	2313.250000	2770.3775	250.500000	20.000000	3.515296	16.064705
max	2023-12-29 00:00:00	2024-01-09 00:00:00	97276.000000	4046.000000	5951.42	567.000000	285.000000	4.967581	19.263804
std	NaN	NaN	28762.687103	1000.995498	1478.699808	145.811981	32.897262	1.013623	4.698091

df.to_csv("Marketing_Campaign_Cleaned.csv", index=False)



Campaign_ID	Campaign_Name	Start_Date	End_Date	Channel	mpression:	Clicks	Spend	Conversions	Active	Campaign_Tag	Conversions_Missing	Start_Date_Missing	Campaign_Duration_Days	CTR_%	Conversion_Rate_%
CMP-00001	Q4_Summer_CMP-00001	24-11-2023	13-12-2023	TikTok	16795	197	102.82	20	TRUE	TI	FALSE	FALSE	19	1.172968	10.15228426
CMP-00002	Q1_Launch_CMP-00002	06-05-2023	12-05-2023	Facebook	1860	30	24.33	1	FALSE	FA	FALSE	FALSE	6	1.612903	3.333333333
CMP-00003	Q3_Winter_CMP-00003	13-12-2023	20-12-2023	Email	77820	843	1323.39	51	FALSE	EM	FALSE	FALSE	7	1.083269	6.049822064
CMP-00004	Q1_BlackFriday_CMP-00004	30-10-2023	03-11-2023	TikTok	55886	2019	2180.38	135	TRUE	TI	FALSE	FALSE	4	3.612712	6.686478455
CMP-00005	Q2_Winter_CMP-00005	22-04-2023	23-04-2023	Facebook	7265	169	252.44	30	TRUE	FA	FALSE	FALSE	1	2.326222	17.75147929
CMP-00006	Q4_BlackFriday_CMP-00006	15-10-2023	28-10-2023	Instagram	83386	2643	2697.03		TRUE	IN	TRUE	FALSE	13	3.169597	
CMP-00007	Q3_Launch_CMP-00007	07-10-2023	23-10-2023	Facebook	38194	1135	1232.76	178	TRUE	FA	FALSE	FALSE	16	2.971671	15.68281938
CMP-00008	Q4_Launch_CMP-00008	23-05-2023	28-05-2023	Instagram	88498	1173	865.7	127	TRUE	IN	FALSE	FALSE	5	1.325454	10.82693947
CMP-00009	Q4_BlackFriday_CMP-00009	23-03-2023	01-04-2023	Google Ads	45131	1179	1046.18	104	TRUE	GO	FALSE	FALSE	9	2.612395	8.821034775
CMP-00010	Q2_Winter_CMP-00010	21-03-2023	01-04-2023	Email	61263	1153	1623.56		FALSE	EM	TRUE	FALSE	11	1.88205	
CMP-00011	Q4_Launch_CMP-00011	22-02-2023	01-03-2023	Instagram	17023	455	523.13	76	TRUE	IN	FALSE	FALSE	7	2.672854	16.7032967
CMP-00012	Q4_Launch_CMP-00012		20-02-2023	Google Ads	42090	763	1267.69	109	FALSE	GO	FALSE	TRUE		1.812782	14.28571429
CMP-00013	Q2_Launch_CMP-00013	16-05-2023	01-06-2023	Google Ads	68221	2975	5292.16	567	FALSE	GO	FALSE	FALSE	16	4.360827	19.05882353
CMP-00014	Q1_Launch_CMP-00014	02-10-2023	15-10-2023	TikTok	65820	1116	1464.72	169	TRUE	TI	FALSE	FALSE	13	1.695533	15.14336918
CMP-00015	Q2_Summer_CMP-00015	10-02-2023	16-02-2023	Google Ads	1769	85	139.18	11	TRUE	GO	FALSE	FALSE	6	4.804975	12.94117647
CMP-00016	Q2_BlackFriday_CMP-00016	13-11-2023	23-11-2023	Email	60735	1617	1552.9	228	TRUE	EM	FALSE	FALSE	10	2.662386	14.10018553
CMP-00017	Q1_Winter_CMP-00017	09-04-2023	01-10-2023	Email	63955	2804	3147.37	536	FALSE	EM	FALSE	FALSE	175	4.384333	19.11554922
CMP-00018	Q1_BlackFriday_CMP-00018	27-04-2023	25-05-2023	Instagram	65925	2317	3635.76	389	TRUE	IN	FALSE	FALSE	28	3.5146	16.78895123
CMP-00019	Q4_Winter_CMP-00019	30-04-2023	18-05-2023	Facebook	68969	2718	2933.59	145	FALSE	FA	FALSE	FALSE	18	3.940901	5.334805004
CMP-00020	Q1_Summer_CMP-00020	21-08-2023	30-08-2023	Email	6311	256	353.99		TRUE	EM	TRUE	FALSE	9	4.056409	
CMP-00021	Q3_BlackFriday_CMP-00021	09-07-2023	17-07-2023	Facebook	84104	2539	1975.49		TRUE	FA	TRUE	FALSE	8	3.018881	
CMP-00022	Q2_BlackFriday_CMP-00022	26-12-2023	02-01-2024	Google Ads	54707	1735	2428.4		TRUE	GO	TRUE	FALSE	7	3.171441	
CMP-00023	Q4_Winter_CMP-00023	08-11-2023	24-11-2023	Google Ads	86305	2637	4726.22	216	FALSE	GO	FALSE	FALSE	16	3.055443	8.19112628
CMP-00024	Q4_Winter_CMP-00024	06-05-2023	01-05-2023	Instagram	29693	995	1793.94	179	FALSE	IN	FALSE	FALSE		3.350958	17.98994975
CMP-00025	Q2_Winter_CMP-00025	24-11-2023	18-12-2023	Email	72932	815	1293.01	157	FALSE	EM	FALSE	FALSE	24	1.117479	19.26380368
CMP-00026	Q1_Winter_CMP-00026	16-06-2023	02-07-2023	Google Ads	94016	2427	3928.42	207	FALSE	GO	FALSE	FALSE	16	2.581475	8.529048208
CMP-00027	Q1_Winter_CMP-00027	28-04-2023	12-05-2023	TikTok	26658	912	1219.24	166	TRUE	TI	FALSE	FALSE	14	3.421112	18.20175439

9.Challenges and Learnings

1. Challenges Faced

During the data collection and preprocessing process, several challenges were identified:

1. Inconsistent Column Names: Some column names contained extra spaces, which needed to be removed for proper data handling.

2. Duplicate Column Names: The dataset contained two columns with the name `Clicks`. The relevant column was retained, while the unnecessary duplicate column was excluded.

3. Missing Values: Some records contained missing values in the `Conversions` column. These values were identified and handled carefully.

4. Inconsistent Categories: The `Channel` column contained an incorrect entry, such as `Gogle`, which was corrected to `Google Ads`.

5. Data Type Conversion: The `Spend` column contained currency symbols, which needed to be removed before converting the values into numeric format.

6. Date Formatting: Different date formats required standardization to make date calculations possible.

7. Feature Creation: Additional columns such as campaign duration, CTR, and conversion rate were created using existing dataset columns.

2. Key Learnings

By completing this task, I gained practical knowledge of the following concepts:

- Understanding and exploring datasets using Pandas.
- Identifying missing values and duplicate records.
- Cleaning inconsistent column names and categorical values.
- Converting data into appropriate formats and data types.
- Working with date and time columns.
- Creating new features using existing data.
- Calculating marketing performance metrics such as CTR and conversion rate.
- Exporting cleaned datasets into CSV format.
- Using Jupyter Notebook for practical data preprocessing.

3. Skills Developed

The task helped me improve my understanding of:

Technical Skills	**Concepts Learned**
Python	Data manipulation
Pandas	Data cleaning and preprocessing
NumPy	Numerical operations
Jupyter Notebook	Interactive coding
Data Analytics	Data preparation and validation

4. Overall Learning Experience

This task provided practical exposure to the initial stages of a data analytics project. I understood that raw data is not always ready for analysis and requires proper cleaning, transformation, and validation.

Working with a marketing campaign dataset helped me apply Python and Pandas concepts to a practical dataset and improved my confidence in handling real-world data.

10.Conclusion

The Week 1 task on Data Collection and Preprocessing provided practical experience in handling and preparing a marketing campaign dataset for further analysis.

During this task, I explored the dataset using Python and Pandas, identified data quality issues, handled missing values, removed unnecessary columns, corrected inconsistent entries, and standardized data types.

I also performed feature engineering by creating additional columns such as Campaign Duration, Click-Through Rate (CTR), and Conversion Rate. These features provide useful information for evaluating marketing campaign performance.

Finally, the processed dataset was validated and exported into CSV format for future analysis.

Overall, this task strengthened my practical understanding of Python, Pandas, NumPy, and the importance of data preprocessing in data analytics.

2. Final Deliverable

The primary deliverable of this task is:

File Name: `Marketing_Campaign_Cleaned.csv`

Description: A cleaned and structured marketing campaign dataset prepared for further analysis and visualization.

3. Acknowledgement

I would like to express my gratitude to the organization and mentors for providing me with this learning opportunity. This task helped me apply theoretical concepts in a practical environment and improve my technical skills in data analytics.

THANK YOU

Week 01 — Data Collection and Preprocessing

Submitted by:

Khushi Singh
B.Tech — Computer Science and Engineering
NGF College of Engineering and Technology